Module Interface Specification for Squigglr

Team #17, Squigglr
Rayan Nasrallah
Ahmed Elzaria
Muhammad Huzaifah
Shamil Canbolat
Moustafa Moustafa

November 13, 2026

1 Revision History

Date	Developer(s)	Change
11 November 2026	Ahmed Elzaria	Added the introduction, the notation, the types shared between modules and the module decomposition, and gave each module its own file (#324).

2 Symbols, Abbreviations and Acronyms

The symbols, abbreviations and acronyms are those of the SRS (Squigglr Team, 2026b) (Section 4, Naming Conventions and Terminology) and of the MG (Squigglr Team, 2026a) (Section 2, Reference Material). Symbolic constants, such as `STAGE_PASS` and `ATTEMPT_LIMIT`, are defined with their units, ranges and defaults in the Symbolic Constants subsection of the SRS. This document adds the following.

Symbol	Description
M	Module, numbered as in the MG (for example, M3)
MIS	Module Interface Specification
D, T, E, I	The stages of a guided lesson: demonstration, full-outline tracing, endpoint-only guidance and independent drawing (SRS Section 9.2)

Contents

1 Revision History i

2 Symbols, Abbreviations and Acronyms ii

3 Introduction 1

4 Notation 1

4.1 Types Shared Between Modules 2

5 Module Decomposition 3

6 MIS of the Input Device Module (M1) 5

7 MIS of the Display Module (M2) 6

8 MIS of the Audio Output Module (M3) 7

9 MIS of the Guided Lesson Control Module (M4) 8

10 MIS of the Lesson Catalogue Module (M5) 9

11 MIS of the Attempt Module (M6) 10

12 MIS of the Attempt Evaluation Module (M7) 11

13 MIS of the Feedback Module (M8) 12

14 MIS of the Improvement Measure Module (M9) 13

15 MIS of the Attempt Record Module (M10) 14

16 MIS of the Creative Drawing Module (M11) 15

17 MIS of the Refinement Control Module (M12) 16

18 MIS of the Child Screen Module (M13) 17

19 MIS of the Adult Access Module (M14) 18

20 MIS of the Learner Profile Module (M15) 19

21 MIS of the Learner Settings Module (M16) 20

22 MIS of the Progress Summary Module (M17) 21

23 MIS of the Adult View Module (M18) **22**

24 MIS of the Stroke Module (M19) **23**

25 MIS of the Geometry Module (M20) **24**

26 MIS of the Edit History Module (M21) **25**

27 MIS of the Refinement Engine Module (M22) **26**

28 MIS of the Storage Module (M23) **27**

29 MIS of the Connection Recovery Module (M24) **28**

30 MIS of the Audit Log Module (M25) **29**

31 Appendix — Generative AI Use Disclosure **31**
31.1 AI Tools Used 31
31.2 How and Where Used 31
31.3 Verification of AI-Generated Content 31

32 Appendix — Reflection **32**

3 Introduction

The following document details the Module Interface Specifications (MIS) for Squigglr, a website on which a young child practises drawing lines, shapes and simple drawings with a stylus. In guided lessons the guidance is removed stage by stage, and in the creative space the child draws freely and can have a drawing refined. A supporting adult chooses the lessons, sets the values that control them and sees the child's progress.

Complementary documents include the System Requirement Specifications (SRS) (Squigglr Team, 2026b) and the Module Guide (MG) (Squigglr Team, 2026a). The MG names each module and the secret it hides; this document gives each module's interface. The full documentation and implementation can be found at https://github.com/AHMEDELZARIA/squigglr.

4 Notation

The structure of the MIS for modules comes from Hoffman and Strooper (1995), with the addition that template modules have been adapted from Ghezzi et al. (2003). The mathematical notation comes from Chapter 3 of Hoffman and Strooper (1995). For instance, the symbol := is used for a multiple assignment statement and conditional rules follow the form $(c_1 \Rightarrow r_1 | c_2 \Rightarrow r_2 | ... | c_n \Rightarrow r_n)$.

The following table summarizes the primitive data types used by Squigglr.

Data Type	Notation	Description
Boolean	$\mathbb{B}$	true or false
character	char	a single symbol or digit
integer	$\mathbb{Z}$	a number without a fractional component in (-∞, ∞)
natural number	$\mathbb{N}$	a number without a fractional component in [1, ∞)
natural number with zero	$\mathbb{N}_0$	a number without a fractional component in [0, ∞)
real	$\mathbb{R}$	any number in (-∞, ∞)

The specification of Squigglr uses some derived data types: sequences, strings, sets, tuples and enumerations. Sequences are lists filled with elements of the same data type, written "seq of T". Strings are sequences of characters. Sets are unordered collections without repeats, written "set of T". Tuples contain a list of values, potentially of different types, and their fields are named, as in $p.x$. An enumeration is a type with a fixed list of values, written as the set of those values, such as $\{D, T, E, I\}$. In addition, Squigglr uses functions, which are defined by the data types of their inputs and outputs. Local functions are described by giving their type signature followed by their specification.

The symbolic constants of the SRS keep their names, written in typewriter font, for example `STAGE_PASS`. A module whose behaviour depends on an adult-set value (the nine values of BR-008 in the SRS) is given the value as an input. Only the Guided Lesson Control Module (M4) reads these values from the Learner Settings Module (M16), when a lesson is chosen, and it passes them on to the modules it calls, so that those modules never read the Learner Settings Module themselves. A constant that the team fixes belongs to the one module that uses it. It is an exported constant of that module when its value is known, as `TRACE_TOL` is, and part of the module's secret while the SRS still leaves its value to be decided.

4.1 Types Shared Between Modules

Table 1 lists the types that appear in the interfaces of more than one module, so that every module uses the same names. They are plain data types, so using one does not make a module use another module. The last column names the module whose data the type describes and which decides its values. StrokeT is the exception: it is an abstract data type, so a module that uses it uses the Stroke Module (M19). A type that one module defines and other modules only name is defined in that module's Exported Types, and the others refer to it there.

Table 1: Types shared between modules

Type	**Definition**	**Owner**
PointT	tuple of $(x : \mathbb{R}, y : \mathbb{R})$, a position on the drawing area	Notation
StrokeT	An abstract data type: the points of one stylus contact, in the order drawn, with the time of each point	M19
PathT	seq of PointT, a line through the points in order. The parts of a lesson's figure are paths.	Notation
DateTimeT	A date and time, in Coordinated Universal Time	Notation
StageT	$\{D, T, E, I\}$, the stages of a guided lesson (the set L of SRS Section 9.2)	M5
DrawingStageT	$\{T, E, I\}$, the stages at which the child draws (the set A of SRS Section 9.2)	M5
LessonIdT	Identifies one lesson in the catalogue	M5
DifficultyT	Identifies one predefined difficulty, such as a missed part or a missed endpoint (BR-007)	M5
FigureT	tuple of (*parts* : seq of PathT, *endpoints* : seq of PointT), the required parts and endpoints of a lesson's figure (SRS Table **??**)	M5

Type	Definition	Owner
ClassT	$\{complete, incomplete, invalid\}$ (the set R of SRS Section 9.2)	M7
GuidanceT	$\{assisted, independent\}$ (the set M of SRS Section 9.2)	M7
EvaluationT	tuple of ($class$: ClassT, $guidance$: GuidanceT, $result$: $\mathbb{R}$, $passed$: $\mathbb{B}$, $difficulties$: set of DifficultyT), the outcome of evaluating the strokes of an attempt. The result is on the 0–100 scale of the SRS.	M7
FeedbackKindT	The kinds of feedback a child can be given after an attempt, such as a passed stage, a missed part or effort on an attempt that did not pass	M8
AdultIdT	Identifies a signed-in supporting adult	M14
LearnerIdT	Identifies one learner profile	M15
SettingsT	tuple of the nine adult-set values of one learner (BR-008), each field named after its SRS constant: *stagePass*, *difficultyStreak*, *attemptLimit*, *improveWindow*, *attemptExtension*, *attemptLimitMax*, *attemptIdle*, *lessonIdlePrompt* and *lessonIdleEnd*	M16

5 Module Decomposition

The following table is taken directly from the Module Guide document for this project (Squiggler Team, 2026a). The module numbers are the same in both documents. The last column names the issue in which each module's interface is written.

Table 2: Module Hierarchy

Level 1	No.	Level 2	MIS part
Hardware-Hiding	M1	Input Device Module	#265
	M2	Display Module	#265
	M3	Audio Output Module	#264
Behaviour-Hiding	M4	Guided Lesson Control Module	#264
	M5	Lesson Catalogue Module	#264
	M6	Attempt Module	#264
	M7	Attempt Evaluation Module	#264
	M8	Feedback Module	#264
	M9	Improvement Measure Module	#264
	M10	Attempt Record Module	#264
	M11	Creative Drawing Module	#265
	M12	Refinement Control Module	#265

Level 1	No.	Level 2	MIS part
	M13	Child Screen Module	#265
	M14	Adult Access Module	#266
	M15	Learner Profile Module	#266
	M16	Learner Settings Module	#266
	M17	Progress Summary Module	#266
	M18	Adult View Module	#266
Software Decision	M19	Stroke Module	#265
	M20	Geometry Module	#264
	M21	Edit History Module	#265
	M22	Refinement Engine Module	#265
	M23	Storage Module	#266
	M24	Connection Recovery Module	#266
	M25	Audit Log Module	#266

Each module's section below follows the same layout: Module, Uses, Syntax (exported constants, exported types and access programs), and Semantics (state variables, environment variables, assumptions, access routine semantics and local functions). In Design Revision -1 the access routine semantics are left for Revision 0. The Uses of each module are those of the uses hierarchy in the MG.

6 MIS of the Input Device Module (M1)

[This module's interface is written in #265. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Types, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

7 MIS of the Display Module (M2)

[This module's interface is written in #265. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Types, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

8 MIS of the Audio Output Module (M3)

[This module's interface is written in #264. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

9 MIS of the Guided Lesson Control Module (M4)

[This module's interface is written in #264. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

10 MIS of the Lesson Catalogue Module (M5)

[This module's interface is written in #264. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

11 MIS of the Attempt Module (M6)

[This module's interface is written in #264. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

12 MIS of the Attempt Evaluation Module (M7)

[This module's interface is written in #264. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

13 MIS of the Feedback Module (M8)

[This module's interface is written in #264. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

14 MIS of the Improvement Measure Module (M9)

[This module's interface is written in #264. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

15 MIS of the Attempt Record Module (M10)

[This module's interface is written in #264. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

16 MIS of the Creative Drawing Module (M11)

[This module's interface is written in #265. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Types, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

17 MIS of the Refinement Control Module (M12)

[This module's interface is written in #265. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Types, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

18 MIS of the Child Screen Module (M13)

[This module's interface is written in #265. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Types, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

19 MIS of the Adult Access Module (M14)

[This module's interface is written in #266. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

20 MIS of the Learner Profile Module (M15)

[This module's interface is written in #266. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

21 MIS of the Learner Settings Module (M16)

[This module's interface is written in #266. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

22 MIS of the Progress Summary Module (M17)

[This module's interface is written in #266. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

23 MIS of the Adult View Module (M18)

[This module's interface is written in #266. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

24 MIS of the Stroke Module (M19)

[This module's interface is written in #265. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Types, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

25 MIS of the Geometry Module (M20)

[This module's interface is written in #264. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

26 MIS of the Edit History Module (M21)

[This module's interface is written in #265. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Types, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

27 MIS of the Refinement Engine Module (M22)

[This module's interface is written in #265. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Types, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

28 MIS of the Storage Module (M23)

[This module's interface is written in #266. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

29 MIS of the Connection Recovery Module (M24)

[This module's interface is written in #266. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

30 MIS of the Audit Log Module (M25)

[This module's interface is written in #266. Follow the layout of the module template: Module, Uses, Syntax (Exported Constants, Exported Access Programs) and Semantics (State Variables, Environment Variables, Assumptions, Access Routine Semantics, Local Functions). —SS]

References

Carlo Ghezzi, Mehdi Jazayeri, and Dino Mandrioli. *Fundamentals of Software Engineering*. Prentice Hall, Upper Saddle River, NJ, USA, 2nd edition, 2003.

Daniel M. Hoffman and Paul A. Strooper. *Software Design, Automated Testing, and Maintenance: A Practical Approach*. International Thomson Computer Press, New York, NY, USA, 1995. URL http://citeseer.ist.psu.edu/428727.html.

Squigglr Team. Module Guide for Squigglr. https://github.com/AHMEDELZARIA/squigglr/blob/main/pdfs/Design/SoftArchitecture/MG.pdf, 2026a.

Squigglr Team. Software Requirements Specification for Squigglr. https://github.com/AHMEDELZARIA/squigglr/blob/main/pdfs/SRS/SRS.pdf, 2026b.

31 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

31.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

31.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

31.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

32 Appendix — Reflection

[Not required for CAS 741 projects —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Problem Analysis and Design.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. Which of your design decisions stemmed from speaking to your client(s) or a proxy (e.g. your peers, stakeholders, potential users)? For those that were not, why, and where did they come from?

4. While creating the design doc, what parts of your other documents (e.g. requirements, hazard analysis, etc), it any, needed to be changed, and why?

5. What are the limitations of your solution? Put another way, given unlimited resources, what could you do to make the project better? (LO_ProbSolutions)

6. Give a brief overview of other design solutions you considered. What are the benefits and tradeoffs of those other designs compared with the chosen design? From all the potential options, why did you select the documented design? (LO_Explores)

7. (After you have implemented another team's module, which means this isn't filled in until after the original deadline). What did you learn by implementing another team's module? Were all the details you needed in the documentation, or did you need to make assumptions, or ask the other team questions? If your team also had another team implement one of your modules, what was this experience like? Are there things in your documentation you could have changed to make the process go more smoothly for when an "outsider" completes some of the implementation?

Your reflection should be about concepts learned in your SE education, such as modular decomposition, cohesion, coupling, information hiding, etc. You should critique the quality and utility of the specification you received.

You should create a PR for your implemented module. A link to the PR should be part of your reflection.